



UNIVERSIDAD
DE GRANADA



Técnicas de los Sistemas Inteligentes

Tema 4: Incertidumbre y Problemas de Decisión Secuenciales

E.T.S. de Ingenierías Informática y de Telecomunicación
Universidad de Granada

*Este documento está protegido por la Ley de Propiedad Intelectual ([Real Decreto Ley 1/1996 de 12 de abril](#)).
Queda expresamente prohibido su uso o distribución sin autorización del autor.*

© Antonio González
gonzalez@ugr.es

Departamento de Ciencias de la
Computación e Inteligencia Artificial
<http://decsai.ugr.es>

Objetivos

- Conocer la importancia de la incertidumbre en el desarrollo de los sistemas inteligentes.
- Conocer como poder resolver problemas de decisión secuencial que incluyen utilidades, incertidumbre y percepción.
- Estos problemas son una generalización de la búsqueda y la planificación.

Estudia el tema en ...

- S. Russell, P. Norvig, Artificial Intelligence: A modern Approach, Tercera Edición, Ed. Pearson, 2010.
- S. Russell, P. Norvig, Artificial Intelligence: A modern Approach, Cuarta Edición, Ed. Pearson, 2022.

Contenido

- Comportamiento bajo incertidumbre
- Problemas de decisión secuenciales
- Relación con el aprendizaje por refuerzo



Comportamiento bajo incertidumbre

- Los agentes casi nunca tienen acceso a toda la verdad sobre su entorno, y, por tanto, tienen que saber comportarse bajo incertidumbre.
- Para un agente es imposible construir una descripción completa y exacta de cómo se desarrollan sus acciones.

Ejemplo

- Plan A_{90} : dejar la casa 90 minutos antes de que salga el vuelo, conducir a velocidad razonable con el coche, aparcar y tomar el vuelo.

¿El plan A_{90} nos llevará al aeropuerto a tiempo?

- El plan A_{90} nos llevará al aeropuerto a tiempo siempre que el coche no se averíe o se quede sin gasolina, y no tenga un accidente, y no haya un accidente u obras en el acceso, y no cancelen el vuelo,...(problema de la cualificación).
- ¿Usamos un plan A_{120} ? Incrementa la creencia del agente en el éxito del plan, pero también la creencia en que aumentará la espera.
- Una solución es dotar al agente de una teoría del mundo simple pero errónea que le permita deducir un plan que funcione en la mayoría de los casos. ¿Cómo? Analizando la importancia relativa de los distintos objetivos, así como de la creencia en que estos se alcanzarán.

Manipulación del conocimiento incierto

$R1: \forall p \text{ Síntoma}(p, \text{Dolor.de.muelas}) \Rightarrow \text{Enfermedad}(p, \text{Caries})$

$R2: \forall p \text{ Síntoma}(p, \text{Dolor.de.muelas}) \Rightarrow \text{Enfermedad}(p, \text{Caries})$
 $\vee \text{Enfermedad}(p, \text{Dolencia.Encias}) \vee \text{Enfermedad}(\text{Absceso}) \vee \dots$

poner en una lista el conjunto completo de antecedentes y consecuentes que se necesitan para asegurar una regla sin excepciones tiene demasiado trabajo y usar tales reglas es bastante difícil.

La ciencia médica no tiene una teoría completa para el dominio.

Problemas

$R\ 3: \quad \forall p \quad Enfermedad(p, Caries) \Rightarrow S\acute{intoma}(p, Dolor.de.muelas)$

- En definitiva, la conexión entre dolor de muelas y caries no es exactamente una consecuencia lógica en ninguna dirección.

Grado de creencia

- El conocimiento del agente puede como mucho proporcionar un grado de creencia en las sentencias relevantes.
- Una opción para representar el grado de creencia es **la teoría de la probabilidad**.
- La probabilidad proporciona una manera de resumir la incertidumbre que se deriva de nuestra pereza e ignorancia.
- Podríamos creer que la regla que relaciona la caries y el dolor de muelas es, por ejemplo, 0.8.
- El 20% restante resume todas las otras causas posibles de dolor de muelas.

Incertidumbre y decisiones racionales

- La presencia de incertidumbre cambia radicalmente el modo en que un agente toma decisiones.

Si creo en lo siguiente:

$$P(A_{25} \text{ llegar a tiempo} \mid \dots) = 0.04$$

$$P(A_{90} \text{ llegar a tiempo} \mid \dots) = 0.70$$

$$P(A_{120} \text{ llegar a tiempo} \mid \dots) = 0.95$$

$$P(A_{1440} \text{ llegar a tiempo} \mid \dots) = 0.9999$$

¿Qué acción elegir?

Principio de la máxima utilidad esperada

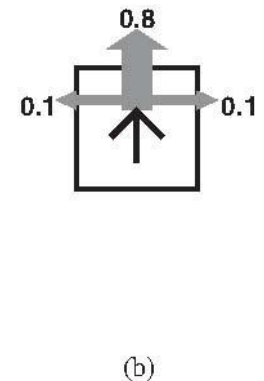
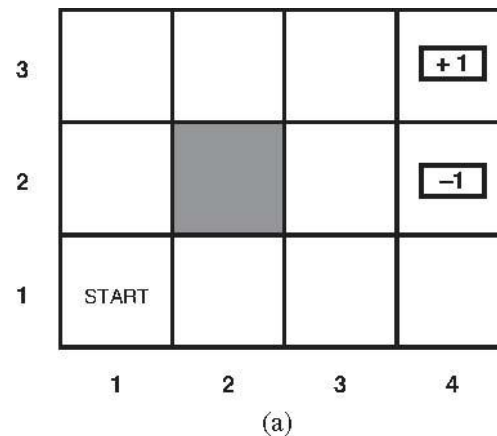
- Para tomar tales elecciones, el agente debe de tener preferencias entre las diferentes consecuencias posibles de los diversos planes.
- Usaremos la teoría de la utilidad para representar y razonar con preferencias.
- La idea fundamental es que el agente es racional si y sólo si elige la acción que le produce la utilidad esperada más alta, en promedio sobre todas las consecuencias de la acción (**Principio de la máxima utilidad esperada**).

Problemas de decisión secuenciales

- Son aquellos en los que la utilidad para el agente depende de la secuencia de acciones.
- Podemos entenderlos como una generalización de la búsqueda y la planificación que hace uso de la interacción con el entorno y la incertidumbre.
 - Incertidumbre asociada a los efectos de las acciones.
 - Incertidumbre asociada a un conocimiento parcial del entorno.

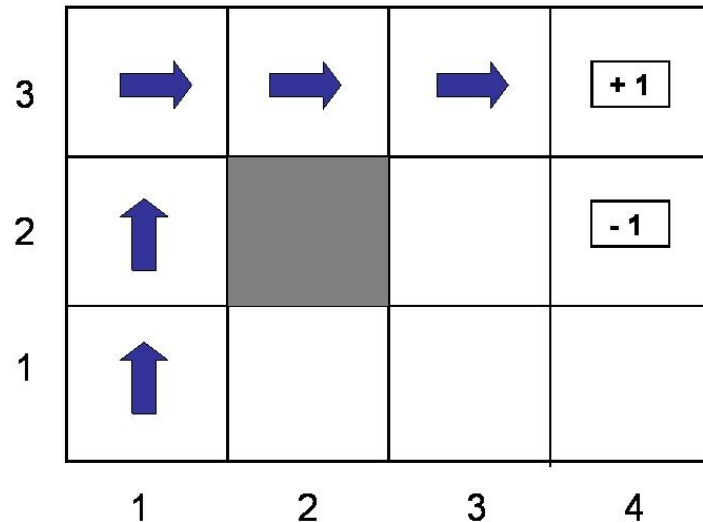
Ejemplo: mundo 4×3 para un robot

- Objetivo: encontrar un camino a la casilla (4,3) (valor +1).
- Entorno completamente observable.
- Estados: (1,1), (1,2), ..., (4,3)
- Acciones: up, down, right, left
 - Movimiento no seguro: probabilidad = 0.8 en la dirección elegida, pero 0.2 en las direcciones perpendiculares a la elegida.
 - Por ejemplo, si acción = up, entonces $p(\text{up}) = 0.8$, $p(\text{left}) = 0.1$ y $p(\text{right}) = 0.1$
- Recompensas: +1 [4,3], -1 [4,2], -0.04 en el resto
- ¿Cuál es la solución?



Ejemplo: mundo 4×3 para un robot

- ¿Es esta una solución?



- Sólo si las acciones son deterministas (no es el caso)
- En este caso tiene prob. de éxito $0.8^5 = 0.32768$
- Solución: encontrar una política, es decir, asignar una acción a cada estado.

Procesos de Decisión de Markov

Un **Proceso de Decisión de Markov (PDM)** es un problema de decisión secuencial para un **entorno completamente observable**, con un modelo de transición de Markov y recompensas aditivas.

- Estados ($s_0, s_1, \dots$), acciones ($a_1, a_2, \dots$), estado inicial (s_0).
- Modelo de Transición: $P(s' | a, s)$ representa la probabilidad de alcanzar s' si se ejecuta la acción a en el estado s .

Por ejemplo: $P([1,2] | \text{up}, [1,1]) = 0.8$.

- Las transiciones son de tipo Markov: la probabilidad de alcanzar s' desde s sólo depende de s , y no de la historia de los estados anteriores.
- Utilidad del agente: depende de la secuencia de estados.
 - Recompensas: $R([4,3]) = +1$, $R([4,2]) = -1$, $R([1,1]) = -0.04$, ...
- Objetivo: Maximizar la recompensa acumulada.

Ejemplo

- Robot recolector de latas:
 - En cada paso, el robot tiene que decidir si debería
 - buscar activamente una lata,
 - esperar que alguien tire una lata, o
 - volver al puesto base para recargar las baterías.
 - Buscar es lo mejor, pero se agotan las baterías; si se queda sin batería mientras busca debe de ser rescatado y eso es malo.
 - Las decisiones se toman en base al nivel actual de energía: *alto, bajo*.
- Recompensa = número de latas recolectadas

Robot Recolector de Latas

$S = \{\text{alto}, \text{bajo}\}$

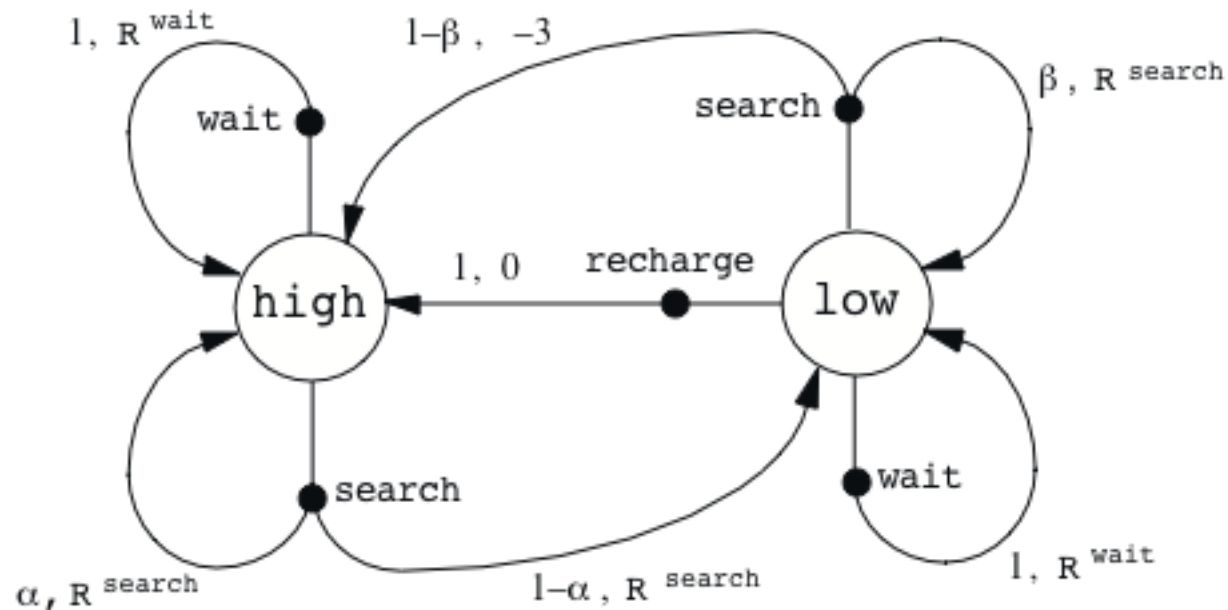
$A(\text{alto}) = \{\text{buscar}, \text{esperar}\}$

$A(\text{bajo}) = \{\text{buscar}, \text{esperar}, \text{recargar}\}$

$R^{\text{busca}} = \text{número esperado de latas mientras busca}$

$R^{\text{espera}} = \text{número esperado de latas mientras espera}$

$R^{\text{busca}} > R^{\text{espera}}$



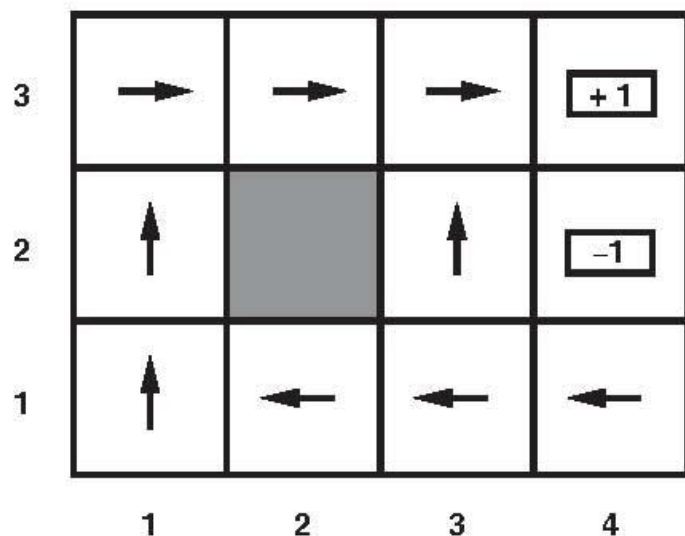
Política

- La solución de un PDM se denomina una política y consiste en asignar a cada estado alcanzable una acción.

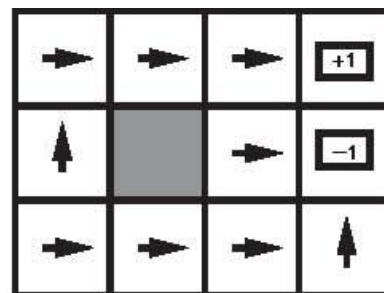
$$\pi : S \rightarrow A$$

- Si el agente tiene una política completa entonces no importa el resultado de una acción, ya que el agente siempre sabe que hacer a continuación.
- Calidad de la política: se mide evaluando la utilidad esperada de los posibles históricos de entorno generados por la política.
- Política óptima π^* : la que tiene mayor utilidad esperada.

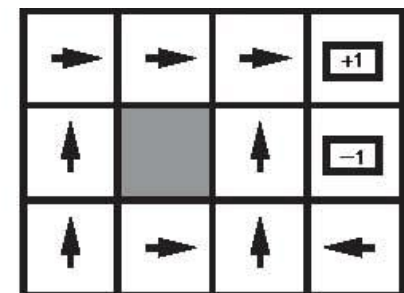
Ejemplo: mundo 4×3 para un robot (soluciones)



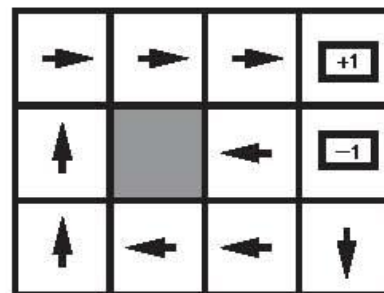
(a)



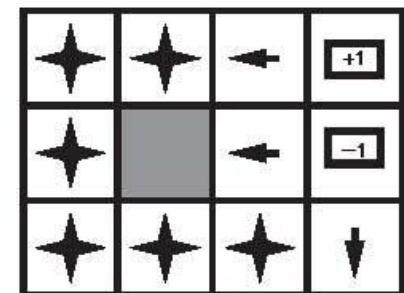
$$R(s) < -1.6284$$



$$-0.4278 < R(s) < -0.0850$$



$$-0.0221 < R(s) < 0$$



$$R(s) > 0$$

(b)

Utilidad para secuencia de acciones

- En el ejemplo hemos medido la utilidad de una secuencia de acciones mediante la suma de la utilidad de cada acción.
- Utilidad de una secuencia: $U_h([s_0, s_1, \dots, s_k])$.
- **Horizonte finito:** existe un tiempo prefijado N tal que después de él no importa nada (el juego se ha acabado), es decir,

$$\forall k \quad U_h([s_0, s_1, \dots, s_N]) = U_h([s_0, s_1, \dots, s_{N+k}])$$

- **Horizonte infinito:** No existe tiempo límite y, por tanto, la acción óptima sólo depende del estado actual (no del tiempo) y la política óptima es estacionaria.
- Con horizonte finito la solución óptima puede cambiar a lo largo del tiempo, por tanto, la política óptima es no estacionaria.
- Con horizonte infinito, la acción óptima depende tan solo del estado actual y la política óptima es estacionaria.

Utilidad para secuencia de acciones

- Nos centraremos en horizonte infinito, por tanto, preferencias estacionarias.
- ¿Cómo calcular la utilidad de secuencias de estados?

- **Recompensas aditivas:**

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$$

- **Recompensas con descuento:**

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

donde γ perteneciente al $[0,1]$ es el factor de descuento y representa la preferencia de un agente por las recompensas actuales frente a las futuras.

Políticas apropiadas

- Si el entorno no tiene estado terminal o nunca lo alcanzamos y usamos recompensas aditivas podemos tener utilidades de $-\infty$ y $+\infty$, lo que es un problema.
- Si usamos recompensas con descuento la utilidad de una secuencia infinita es finita:

$$U_h([s_0, s_1, s_2, \dots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1-\gamma}$$

- **Política apropiada:** aquella que garantiza al agente alcanzar un estado terminal.
 - En este caso podemos usar recompensas aditivas o con descuento.
- **Políticas no apropiadas.** No garantizan llegar a un estado terminal.
 - Tan solo podríamos usar recompensas con descuento.

Política óptima

- La **utilidad esperada de una política** π es la suma esperada de las recompensas con descuento obtenidas sobre todas las posibles secuencias de estados que pueden darse cuando se ejecuta π .

$$U^{\pi}(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t) \right]$$

- La **política óptima** π^* viene dada por:

$$\pi_s^* = \operatorname{argmax}_{\pi} U^{\pi}(s)$$

Utilidad de los estados

- Idea básica para calcular la política óptima: **calcular la utilidad de cada estado y usar dichas utilidades para seleccionar la acción óptima en cada estado.**
- La utilidad de un estado ($U(s)$) es la utilidad esperada de las secuencias de estados que le pueden seguir.
- Como las secuencias dependen de la política usada, definimos $U^\pi(s)$ para una política concreta π .

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \mid \pi, s_0 = s \right]$$

- Por tanto, la utilidad real de un estado $U(s)$ es justamente $U^{\pi^*}(s)$.
- Nótese la diferencia entre $U(s)$ y $R(s)$:
 - $R(s)$: recompensa a corto plazo
 - $U(s)$: recompensa total a largo plazo

Utilidad de los estados: ejemplo

Gamma=1.0

3	0.812	0.869	0.919	+ 1
2	0.770		0.675	- 1
1	0.702	0.676	0.593	0.406
	1	2	3	4

Gamma=0.9

3	0.566	0.732	0.882	+ 1
2	0.445		0.551	- 1
1	0.331	0.234	0.185	0.002

Cálculo de la política óptima

- A partir de $U(s)$ y del principio de máxima utilidad esperada el agente puede elegir la acción que maximice la utilidad esperada del estado resultante:

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' \mid s, a) U(s')$$

Ecuación de Bellman

- Utilizando las dos expresiones para la política óptima π^* que hemos visto, se obtiene que hay una relación directa entre la utilidad de un estado y la de sus vecinos.
- La **ecuación de Bellman** expresa esta relación:

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U(s')$$

Ejemplo

- Por ejemplo, la ecuación para el estado (1,1):

$$U(1,1) = -0.04 + \gamma \max \begin{cases} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1), & (Up) \\ 0.9U(1,1) + 0.1U(1,2), & (Left) \\ 0.9U(1,1) + 0.1U(2,1), & (Down) \\ 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1) \end{cases} \quad (Right)$$

- Si hay n estados, entonces tenemos n ecuaciones cada una con n incógnitas (las utilidades de los n estados).
- Problema: las ecuaciones son no lineales

Algoritmo de iteración de valores

- Una posible solución es utilizar una aproximación iterativa:
- Empezamos con una asignación arbitraria para los valores de utilidad de los estados, calculamos la parte de la derecha de la ecuación de Bellman y la asignamos a la parte izquierda, se repite el proceso hasta alcanzar un equilibrio:
- Si $U_i(s)$ es la utilidad de s en la iteración i , entonces aplicamos la actualización de Bellman, que consiste en:

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U_i(s')$$

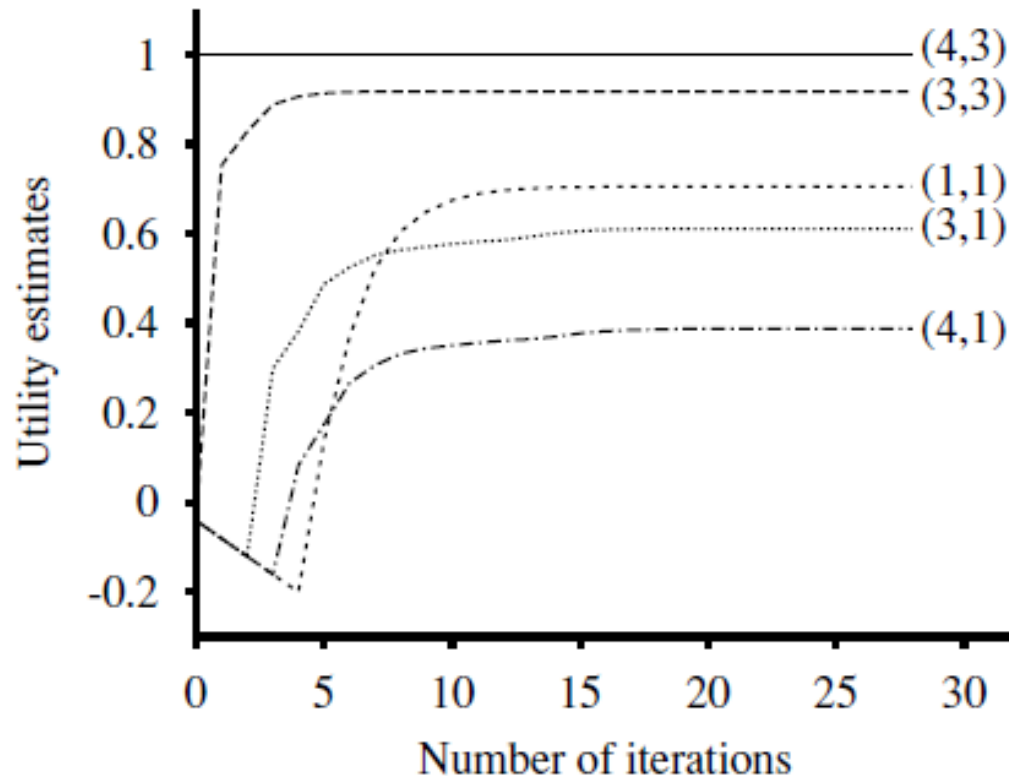
Iteración de valores: algoritmo

- Si se aplica la actualización de Bellman indefinidamente se garantiza llegar a un estado de equilibrio en el que los valores obtenidos son las soluciones al sistema de ecuaciones de Bellman y la política asociada es la óptima.

```
function VALUE-ITERATION( $mdp, \epsilon$ ) returns a utility function
  inputs:  $mdp$ , an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ ,
           rewards  $R(s)$ , discount  $\gamma$ 
            $\epsilon$ , the maximum error allowed in the utility of any state
  local variables:  $U, U'$ , vectors of utilities for states in  $S$ , initially zero
                     $\delta$ , the maximum change in the utility of any state in an iteration

  repeat
     $U \leftarrow U'; \delta \leftarrow 0$ 
    for each state  $s$  in  $S$  do
       $U'[s] \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s']$ 
      if  $|U'[s] - U[s]| > \delta$  then  $\delta \leftarrow |U'[s] - U[s]|$ 
  until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
  return  $U$ 
```

Iteración de valores: Ejemplo



Búsqueda de π^* por Iteración de Políticas

- Idea: si una acción es claramente mejor que todas las restantes entonces las magnitudes exactas de las utilidades de los estados involucradas no necesitan ser precisas.
- El algoritmo de iteración de políticas parte de una política inicial π_0 y alterna los siguientes pasos:
 1. Evaluación de la política: dada π_i calcular $U_i = U^{\pi_i}$, es decir, la utilidad de cada estado si π_i fuese ejecutada
 2. Mejora de la política: calcular la nueva política π_{i+1} basándonos en el principio de máxima utilidad esperada y considerando U_i .
- El algoritmo concluye cuando no hay mejora, entonces el resultado es una solución a las ecuaciones de Bellman y, por tanto, π_i es una política óptima.

Iteración de políticas: fase de evaluación

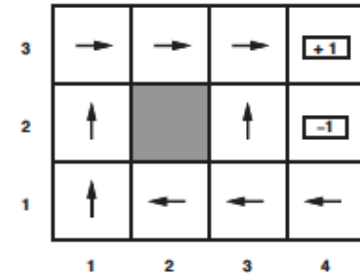
- La fase de mejora de la política es sencilla, pero ¿y la evaluación de la política?
- Utilizamos una versión simplificada de la ecuación de Bellman:

$$U_i(s) = R(s) + \gamma \sum_{s'} P(s' | s, \pi_i(s)) U_i(s')$$

- Por ejemplo, siguiendo la política:

$$U_i([1,1]) = -0.04 + 0.8U_i([1,2]) + 0.1U_i([1,1]) + 0.1U_i([2,1])$$

$$U_i([1,2]) = -0.04 + 0.8U_i([1,3]) + 0.2U_i([1,2]) \dots$$



- En este caso tenemos n ecuaciones lineales con n incógnitas que pueden ser resueltas con un método exacto del álgebra lineal, con complejidad $O(n^3)$.

Iteración de políticas: fase de evaluación

- Cuando n es grande $O(n^3)$ puede ser demasiado tiempo, y es conveniente utilizar la actualización de Bellman simplificada:

$$U_{i+1}(s) \leftarrow R(s) + \gamma \sum_{s'} P(s' | s, \pi_i(s)) U_i(s')$$

- este paso se repite un número fijo de veces para producir la siguiente estimación de los valores de utilidad (**Algoritmo de iteración de políticas modificada**).

Iteración de políticas: algoritmo

function POLICY-ITERATION(mdp) **returns** a policy

inputs: mdp , an MDP with states S , actions $A(s)$, transition model $P(s' | s, a)$

local variables: U , a vector of utilities for states in S , initially zero

π , a policy vector indexed by state, initially random

repeat

$U \leftarrow \text{POLICY-EVALUATION}(\pi, U, mdp)$

$unchanged? \leftarrow \text{true}$

for each state s **in** S **do**

if $\max_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s'] > \sum_{s'} P(s' | s, \pi[s]) U[s']$ **then do**

$\pi[s] \leftarrow \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s']$

$unchanged? \leftarrow \text{false}$

until $unchanged?$

return π

Procesos de decisión de Markov parcialmente observables

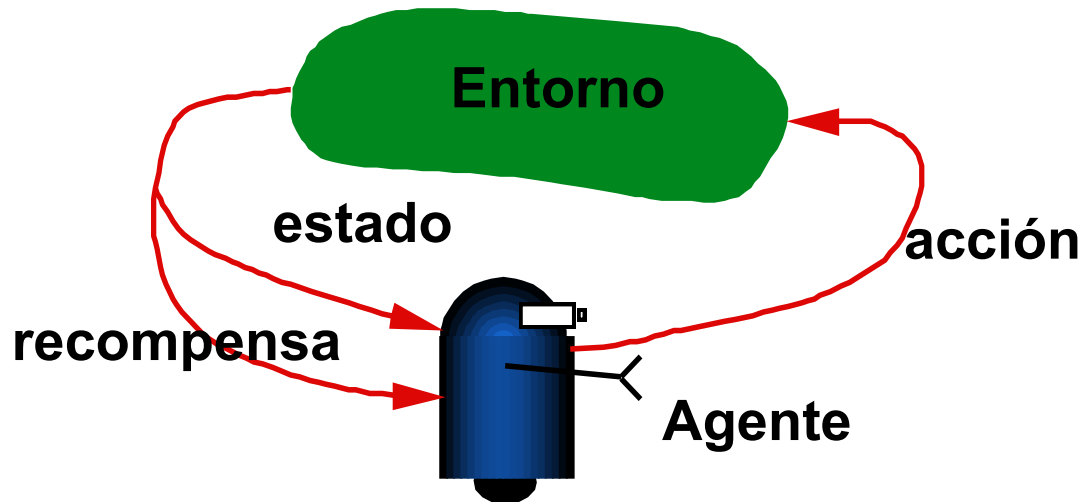
- La descripción de los procesos de decisión de Markov hecha hasta ahora asume que el entorno es completamente observable.
- Cuando el estado es parcialmente observable:
 - El agente no conoce necesariamente en qué estado se encuentra,
 - y no puede ejecutar la acción recomendada para ese estado.

Relación con el aprendizaje por refuerzo

- ¿Y cuando no tenemos un modelo?
- Hemos visto como obtener π^* en un PDM, pero ¿qué ocurre si no existe especificación de las recompensas $R(s)$ ni del modelo de transición $P(s|a,s')$?
- En este caso queremos que el agente aprenda a comportarse bien en un mundo definido por un PDM, pero en el que de inicio no sabemos nada sobre R y P (Aprendizaje por refuerzo).
- Necesita describir:
 - Cómo cambiar la política a partir de la experiencia.
 - Cómo explorar el entorno.
- Técnicas de aprendizaje:
 - Aprendizaje por refuerzo pasivo (π está fija, aprender U)
 - Aprendizaje por refuerzo activo (aprende π y U)

Recompensas

- El agente recibe la recompensa como parte de su entrada.
- El aprendizaje por refuerzo es más general que el aprendizaje supervisado y no supervisado.
- El agente aprende de la interacción con el entorno para alcanzar un objetivo.



Estimación Directa de la Utilidad

- Objetivo: sin conocer R ni P, pero con la política π fijada estimar las utilidades U.
- Posibilidad: el agente usando su política π lanza una serie de pruebas y almacena el resultado:

prueba 1: $(1, 1)_{-0,04} \rightarrow (1, 2)_{-0,04} \rightarrow (1, 3)_{-0,04} \rightarrow (1, 2)_{-0,04} \rightarrow$
 $(1, 3)_{-0,04} \rightarrow (2, 3)_{-0,04} \rightarrow (3, 3)_{-0,04} \rightarrow (4, 3)_{+1}$

prueba 2: $(1, 1)_{-0,04} \rightarrow (1, 2)_{-0,04} \rightarrow (1, 3)_{-0,04} \rightarrow (2, 3)_{-0,04} \rightarrow$
 $(3, 3)_{-0,04} \rightarrow (3, 2)_{-0,04} \rightarrow (3, 3)_{-0,04} \rightarrow (4, 3)_{+1}$

prueba 3:

$(1, 1)_{-0,04} \rightarrow (2, 1)_{-0,04} \rightarrow (3, 1)_{-0,04} \rightarrow (3, 2)_{-0,04} \rightarrow (4, 2)_{-1}$

- Nótese que, al ser un entorno totalmente observable, tras una acción el agente percibe el nuevo estado y la recompensa obtenida.

Estimación Directa de la Utilidad

- La idea es que la utilidad de un estado es la recompensa total esperada del estado en adelante, y cada prueba proporciona un ejemplo del valor para cada estado visitado.
- Por ejemplo, la prueba 1 anterior proporciona:

Estado	(1,1)	(1,2)	(1,3)	(1,2)	(1,3)	(2,3)	(3,3)
Valor	0.72	0.76	0.80	0.84	0.88	0.92	0.96

- Este método tiene éxito reduciendo un problema de AR a uno de aprendizaje supervisado.
- Pero pierde una fuente muy importante de información: las utilidades no son independientes, la utilidad de un estado es igual a la propia recompensa más la utilidad esperada de sus estados sucesores (ecuación de Bellman).
- Esto provoca que la estimación directa busca en un espacio de hipótesis para U que es más grande de lo necesario, provocando una convergencia muy lenta.

Programación dinámica adaptativa

- Para aprovechar las restricciones entre los estados, hay que aprender cómo están conectados, es decir, el modelo de transiciones.
- Un agente de programación dinámica adaptativa (PDA) trabaja aprendiendo el modelo de transiciones del entorno a medida que las recorre, y resolviendo el correspondiente PDM mediante un método de programación dinámica.
- Por tanto, hay que introducir el modelo aprendido $P(s' | s, \pi(s))$ y la recompensa observada $R(s)$ en las ecuaciones de Bellman:

$$U^\pi(s) = R(s) + \gamma \sum_{s'} P(s' | s, \pi(s)) U^\pi(s')$$

- Como dichas ecuaciones son lineales al conocer la política se puede usar algún paquete de álgebra lineal.
- Otra opción es usar el algoritmo de iteración de políticas modificado, usando un proceso simplificado de iteración del valor para actualizar la utilidad estimada después de cada cambio en el modelo aprendido.
- Aprender el modelo es sencillo: cálculo de frecuencias.

Aprendizaje por diferencia temporal

- Toma lo mejor de los dos enfoques anteriores: aproximar las ecuaciones de restricciones sin resolverlas para todos los posibles estados.
- La clave es usar las transiciones observadas para ajustar los valores de los estados observados para que sean acordes con las ecuaciones de restricciones.
- Ejemplo: en la prueba 1

$(1,1)_{-0.04} \rightarrow (1,2)_{-0.04} \rightarrow (1,3)_{-0.04} \rightarrow (1,2)_{-0.04} \rightarrow (1,3)_{-0.04} \rightarrow (2,3)_{-0.04} \rightarrow (3,3)_{-0.04} \rightarrow (4,3)_{+1}$

teníamos $U^\pi((1,3)) = 0.84$ y $U^\pi((2,3)) = 0.92$.

- Entonces, si esa transición ocurriese todo el tiempo, esperaríamos: $U^\pi((1,3)) = -0.04 + U^\pi((2,3))$, con lo que $U^\pi((1,3))$ sería 0.88, es decir, su estimación (0.84) es baja y deberíamos subirla.

Aprendizaje por diferencia temporal

- En general, cuando hay una transición de s a s' actualizamos con:

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha(R(s) + \gamma U^\pi(s') - U^\pi(s))$$

α es el parámetro de la tasa de aprendizaje.

- El nombre de diferencia temporal viene del uso de la diferencia de utilidades entre estados sucesivos

Comparación de enfoques

- Una diferencia es que ADT ajusta un estado para que esté de acuerdo con su sucesor observado, mientras que PDA ajusta el estado para que esté de acuerdo con todos los sucesores que pudieran ocurrir ponderados por sus probabilidades.
- Otra diferencia es que ADT hace un único ajuste por cada transición observada, mientras que PDA hace tantos como necesite para restaurar la consistencia entre la utilidad estimada U y el modelo del entorno P .
- En definitiva, ADT se puede ver como una tosca, pero eficiente, primera aproximación de PDA.

Aprendizaje por refuerzo activo

- El aprendizaje-Q es un método libre de modelo, ya que un agente de diferencia temporal que aprende una función-Q no necesita un modelo de P ni para el aprendizaje ni para la selección de acciones.
- La ecuación en este caso es: $U(s) = \max_a Q(a,s)$

$$Q(s, a) = R(s) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q(s', a')$$

- La ecuación actualizada, usando el aprendizaje por diferencia temporal, es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (R(s) + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

y se calcula siempre que se ejecuta a en s conduciendo a s'.